

Library Management System — Project Reference

Paste this entire file into any new Claude session to resume instantly.

Say: "Here is my project reference. I need help continuing my C++ Library Management System."

Project Overview

A terminal-based C++ Library Management System with an optional ncurses TUI.

Demonstrates: OOP, inheritance, file I/O, vectors, operator overloading, exception handling, templates.

Compiler: g++ with C++17 or later

Platform: macOS / Linux

TUI Library: ncurses (optional, layer on after core logic works)

File Structure

```
library-management/  
├── main.cpp  
├── src/  
│   ├── book.cpp  
│   ├── user.cpp  
│   ├── member.cpp  
│   ├── admin.cpp  
│   └── library.cpp  
├── include/  
│   ├── book.h  
│   ├── user.h  
│   ├── member.h  
│   ├── admin.h  
│   └── library.h  
└── data/  
    ├── books.txt  
    └── members.txt
```

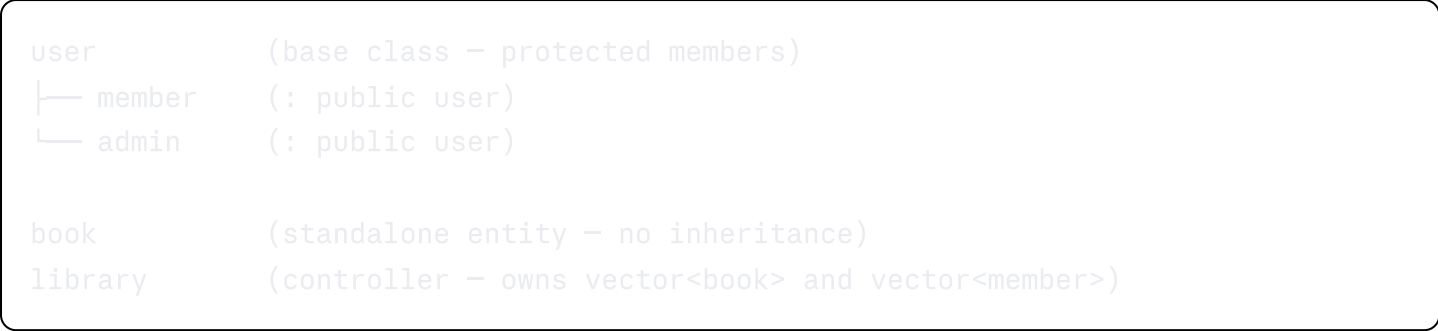
Compile command:

```
bash
```

```
g++ -std=c++17 -I include src/*.cpp main.cpp -o library
```

Class Architecture

Inheritance Tree



Class: `user` (base class)

File: `include/user.h` / `src/user.cpp`

Attributes (protected)

Name	Type	Notes
name	string	
contactNumber	string	use string not int — numbers can start with 0

Methods (public)

Signature	Notes
<code>user()</code>	default constructor
<code>user(string, string)</code>	parameterized constructor
<code>string get_name()</code>	getter
<code>string get_contactNumber()</code>	getter
<code>void set_name(string)</code>	setter
<code>void set_contactNumber(string)</code>	setter

Known bug in current code: `user.cpp` methods are missing `user::` scope prefix — they are free functions, not class methods. Fix: add `user::` before each definition.

Known bug in current code: In `user.h`, constructors are declared as `user::user()` inside the class body — remove the `user::` prefix inside the header. That prefix only belongs in the `.cpp`

file.

Known bug in current code: Setters were declared as returning `string`/`int` instead of `void`. Setters always return `void` and take a parameter.

Class: `member` (inherits `user`)

File: `include/member.h` / `src/member.cpp`

Attributes (private)

Name	Type	Notes
<code>memberId</code>	<code>int</code>	unique ID
<code>borrowedBooks</code>	<code>vector<int></code>	stores ISBNs or book IDs of currently borrowed books

Note: `nborrowedBooks` is unnecessary — use `borrowedBooks.size()` instead.

Methods (public)

Signature	Notes
<code>member()</code>	default constructor
<code>member(string, string, int)</code>	name, contact, memberId — <code>borrowedBooks</code> defaults to empty vector
<code>int get_memberId()</code>	getter
<code>vector<int> get_borrowedBooks()</code>	returns the full vector
<code>void set_memberId(int)</code>	setter
<code>void addBorrowedBook(int bookId)</code>	push_back onto <code>borrowedBooks</code> vector
<code>void removeBorrowedBook(int bookId)</code>	find and erase from vector
<code>float calculateFine(int daysOverdue)</code>	returns fine amount — fine rate is a constant (e.g. ₹2/day)

Missing include in current code

`member.h` uses `vector<int>` but is missing `#include <vector>`. Add it.

Known bug in current code (member.cpp constructor)

The if/else block attempting to initialize `borrowedBooks` is invalid C++. Since `borrowedBooks` is a `vector`, it is already default-initialized to empty. Just assign `memberId = pmemberId` and nothing else for the vector.

Class: `admin` (inherits `user`)

File: `include/admin.h` / `src/admin.cpp`

Attributes (private)

Name	Type	Notes
<code>adminId</code>	<code>int</code>	
<code>password</code>	<code>string</code>	store hashed ideally, plain string for project scope
<code>isAuthenticated</code>	<code>bool</code>	default false

Methods (public)

Signature	Notes
<code>admin()</code>	default constructor
<code>admin(string, string, int, string)</code>	name, contact, adminId, password
<code>int get_adminId()</code>	getter
<code>bool get_isAuthenticated()</code>	getter
<code>bool authenticate(string inputPassword)</code>	compares input to stored password, sets <code>isAuthenticated</code> , returns bool

Known bugs in current code

- Include guard mismatch: `#ifndef ADMIN_H` but `#define ADMIN_h` — the `(h)` must be uppercase `(H)`
 - `#include<user.h>` should be `#include "user.h"` (quotes for your own files, angle brackets for system libraries)
 - Setter and getter method bodies are all empty — need implementations
-

Class: `book` (standalone)

File: `include/book.h` / `src/book.cpp`

Attributes (private)

Name	Type	Notes
title	string	
author	string	
genre	string	
isbn	string	use string — ISBNs have leading zeros and hyphens
totalCopies	int	
availableCopies	int	

Note: `overDue` was in the original code but belongs in `member` or tracked via a transaction, not on the book itself. Remove it from `book`.

Methods (public)

Signature	Notes
<code>book()</code>	default constructor
<code>book(string, string, string, string, int, int)</code>	parameterized
getters for all 6 fields	implemented and working in book.cpp
setters for all 6 fields	implemented and working in book.cpp
<code>friend ostream& operator<<(ostream&, const book&)</code>	for clean cout printing of a book

book.cpp is the cleanest file — mostly correct, no major bugs.

Class: `library` (controller)

File: `include/library.h` / `src/library.cpp`

Attributes (private)

Name	Type	Notes
books	vector<book>	all books in the system
members	vector<member>	all registered members

Critical bug in current code: `library.h` stores `reqMem_id`, `name`, `cnctNo`, `brdBooks` as its attributes — these are member-level data, not library-level. Replace all of them with the two vectors above.

Critical bug in current code: Several setters are named `get_` instead of `set_`. Rename them.

Methods (public)

Book Management

Signature	Notes
<code>void addBook(book b)</code>	push_back onto books vector
<code>void removeBook(string isbn)</code>	find by isbn, erase from vector
<code>void updateBook(string isbn)</code>	find book, call setters on it
<code>void searchBook(string query)</code>	search by title or author, print matches
<code>void displayAllBooks()</code>	loop through books vector, print each
<code>void displayAvailableBooks()</code>	loop, print only where availableCopies > 0
<code>void displayBorrowedBooks()</code>	loop, print only where availableCopies < totalCopies
<code>void displayOverdueBooks()</code>	requires due date tracking — see notes
<code>void getMostBorrowedBooks()</code>	sort or count borrow frequency

Member Management

Signature

Notes

<code>void registerMember(member m)</code>	push_back onto members vector
<code>void removeMember(int memberId)</code>	find by ID, erase
<code>void searchMember(string name)</code>	find by name, print
<code>void displayAllMembers()</code>	loop through members vector, print each

Borrow & Return

Signature

Notes

<code>void issueBook(int memberId, string isbn)</code>	find member + book, decrement availableCopies, call member.addBorrowedBook()
<code>void returnBook(int memberId, string isbn)</code>	find member + book, increment availableCopies, call member.removeBorrowedBook()
<code>void renewBook(int memberId, string isbn)</code>	extend due date if no reservation
<code>void payFine(int memberId)</code>	call member.calculateFine(), record payment

Persistence

Signature

Notes

<code>void saveBooksToFile()</code>	write books vector to data/books.txt
<code>void loadBooksFromFile()</code>	read data/books.txt, reconstruct books vector
<code>void saveMembersToFile()</code>	write members vector to data/members.txt
<code>void loadMembersFromFile()</code>	read data/members.txt, reconstruct members vector

Extras

Signature

Notes

<code>template<typename T> T search(vector<T>& vec, string query)</code>	generic search working on both books and members
--	--

File Format (data/)

data/books.txt

One book per line, pipe-separated:

```
title|author|genre|isbn|totalCopies|availableCopies
```

Example:

```
The Pragmatic Programmer|David Thomas|Technology|9780135957059|3|2  
Atomic Habits|James Clear|Self-Help|9780735211292|5|5
```

Bug in current books.txt: The format has `(id | author | genre | tcopies | acopies)` — missing `(title)`. Fix the format to match above.

data/members.txt

One member per line, pipe-separated:

```
memberId|name|contactNumber|borrowedBookId1,borrowedBookId2
```

If no borrowed books, last field is empty:

```
1001|Soham|9876543210|  
1002|Aayush|9123456789|9780135957059,9780735211292
```

How File I/O Works (concept)

Saving: Loop through the vector. For each object, write all fields joined by `(|)` as one line. Use `(ofstream)`.

Loading: Open file with `(ifstream)`. Read line by line with `(getline)`. For each line, split on `(|)` using `(stringstream)` or `(find)+(substr)`. Convert string pieces back to their types (`(stoi)` for int). Construct the object and `push_back` into the vector.

Load is called once at program start. Save is called on exit (or after every write operation).

Key Design Decisions

- `(Library)` never touches private data of `(book)` or `(member)` directly — it always calls their

public getters/setters

- `Admin` authenticates via `authenticate()` — once authenticated, it calls `Library` functions. `Admin` is NOT a duplicate of `Library`'s functions
 - `contactNumber` is `string` not `int` — phone numbers can have leading zeros
 - `isbn` is `string` not `int` — ISBNs have hyphens
 - `borrowedBooks` in `member` stores ISBNs (strings) so library can look up the book object when needed
-

Progress Status

File	Status
book.h / book.cpp	Mostly done — working getters/setters, add operator<<
user.h / user.cpp	Has bugs — fix constructor declarations and scope prefixes
member.h / member.cpp	Has bugs — fix constructor, add vector include
admin.h / admin.cpp	Has bugs — fix include guard, include path, empty method bodies
library.h / library.cpp	Needs redesign — replace attributes with vectors, implement all methods
main.cpp	Skeleton only — vectors demonstrated, include lines commented out
data/books.txt	Wrong format — missing title column
data/members.txt	Empty

Next Steps (in order)

1. Fix `user.h` and `user.cpp` (constructor syntax, scope prefix, setter signatures)
2. Fix `member.h` (add vector include) and `member.cpp` (fix constructor)
3. Fix `admin.h` (include guard, include path) and `admin.cpp` (implement methods)
4. Redesign `library.h` (replace attributes with vectors, correct method signatures)
5. Implement `library.cpp` methods one by one starting with `addBook/displayAllBooks`
6. Implement file I/O (`saveBooksToFile` / `loadBooksFromFile`)
7. Wire everything together in `main.cpp`
8. Add ncurses TUI on top once core logic works